Lab 7: Fuzz Testing for Input Validation Bugs

Repository: https://github.com/sanchita-suni/L6_L7_54_museum_admission

SS1: Original application

python museum_admission_parser.py

```
/workspace/L6_L7_54_museum_admission/LAB7$ python museum_admission_parser.py
==== Running format_visitor_name ====
Enter a visitor's full name (e.g. asha rao): asha
Name on ticket: Asha,

==== Running parse_group_ages ====
Enter the ages in the group separated by spaces (e.g. 34 12 70): 34 twelve 70
Traceback (most recent call last):
  File "/workspace/L6_L7_54_museum_admission/LAB7/museum_admission_parser.py", line 38, in <module>
    main()
    ~~~~^^
  File "/workspace/L6_L7_54_museum_admission/LAB7/museum_admission_parser.py", line 31, in main
    print("Group ages:", parse_group_ages(value))
                        ~~~~~~~~~~~~~~~~^^^^^^^
  File "/workspace/L6_L7_54_museum_admission/LAB7/museum_admission_parser.py", line 14, in parse_group_ages
    return [int(age) for age in text.split()]
            ~~~^^^^^
ValueError: invalid literal for int() with base 10: 'twelve'
/workspace/L6_L7_54_museum_admission/LAB7$
```

SS2: Fuzz tests failing on the original application

pytest test_museum_admission_parser.py (full output across three screenshots)

```
/workspace/L6_L7_54_museum_admission/LAB7$ pytest test_museum_admission_parser.py
============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-9.1.1, pluggy-1.6.0
rootdir: /workspace/L6_L7_54_museum_admission/LAB7
plugins: platformdirs-4.13.3, anyio-4.15.1, hypothesis-6.170.15
collected 3 items

test_museum_admission_parser.py FFF                                      [100%]

=================================== FAILURES ===================================
_____________________ test_format_visitor_name_no_crash ______________________
  + Exception Group Traceback (most recent call last):
  |   File "/workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py", line 6, in test_format_visitor_name_no_crash
  |     def test_format_visitor_name_no_crash(s):
  |                    ^^^
  |   File "/usr/local/lib/python3.13/site-packages/hypothesis/core.py", line 2334, in wrapped_test
  |     raise the_error_hypothesis_found
  | ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py", line 8, in test_format_visitor_name_no_crash
    |     result = format_visitor_name(s)
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/museum_admission_parser.py", line 7, in format_visitor_name
    |     return parts[-1].title() + ', ' + ' '.join(parts[:-1]).title()
    |            ~~~~~^^^^
    | IndexError: list index out of range
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py", line 10, in test_format_visitor_name_no_crash
    |     assert False, f"format_visitor_name crashed on {s!r}: {type(e).__name__}: {e}"
    | AssertionError: format_visitor_name crashed on '': IndexError: list index out of range
    | assert False
    | Failing test case: test_format_visitor_name_no_crash(
    |     s='',
    | )
    | Explanation:
    |     These lines were always and only run by failing test cases:
    |         /workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py:9
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py", line 8, in test_format_visitor_name_no_crash
    |     result = format_visitor_name(s)
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/museum_admission_parser.py", line 6, in format_visitor_name
    |     parts = name.split()
    |             ^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'split'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py", line 10, in test_format_visitor_name_no_crash
    |     assert False, f"format_visitor_name crashed on {s!r}: {type(e).__name__}: {e}"
    | AssertionError: format_visitor_name crashed on None: AttributeError: 'NoneType' object has no attribute 'split'
    | assert False
    | Failing test case: test_format_visitor_name_no_crash(
    |     s=None,
    | )
    | Explanation:
    |     These lines were always and only run by failing test cases:
    |         /workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py:9
    +------------------------------------
```

```
_____________________ test_parse_group_ages_no_crash _____________________
  + Exception Group Traceback (most recent call last):
  |   File "/workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py", line 16, in test_parse_group_ages_no_crash
  |     def test_parse_group_ages_no_crash(s):
  |                ^^^
  |   File "/usr/local/lib/python3.13/site-packages/hypothesis/core.py", line 2334, in wrapped_test
  |     raise the_error_hypothesis_found
  | ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py", line 18, in test_parse_group_ages_no_crash
    |     result = parse_group_ages(s)
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/museum_admission_parser.py", line 14, in parse_group_ages
    |     return [int(age) for age in text.split()]
    |             ~~~^^^^^
    | ValueError: invalid literal for int() with base 10: ':'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py", line 20, in test_parse_group_ages_no_crash
    |     assert False, f"parse_group_ages crashed on {s!r}: {type(e).__name__}: {e}"
    | AssertionError: parse_group_ages crashed on ':': ValueError: invalid literal for int() with base 10: ':'
    | assert False
    | Failing test case: test_parse_group_ages_no_crash(
    |     s=':',
    | )
    | Explanation:
    |     These lines were always and only run by failing test cases:
    |         /workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py:19
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py", line 18, in test_parse_group_ages_no_crash
    |     result = parse_group_ages(s)
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/museum_admission_parser.py", line 14, in parse_group_ages
    |     return [int(age) for age in text.split()]
    |                               ^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'split'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py", line 20, in test_parse_group_ages_no_crash
    |     assert False, f"parse_group_ages crashed on {s!r}: {type(e).__name__}: {e}"
    | AssertionError: parse_group_ages crashed on None: AttributeError: 'NoneType' object has no attribute 'split'
    | assert False
    | Failing test case: test_parse_group_ages_no_crash(
    |     s=None,
    | )
    | Explanation:
    |     These lines were always and only run by failing test cases:
    |         /workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py:19
    +-----------------------------------
_____________________ test_parse_visit_date_no_crash _____________________
  + Exception Group Traceback (most recent call last):
  |   File "/workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py", line 26, in test_parse_visit_date_no_crash
  |     def test_parse_visit_date_no_crash(s):
  |                ^^^
  |   File "/usr/local/lib/python3.13/site-packages/hypothesis/core.py", line 2334, in wrapped_test
  |     raise the_error_hypothesis_found
  | ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
```

```
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py", line 28, in test_parse_visit_date_no_crash
    |     result = parse_visit_date(s)
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/museum_admission_parser.py", line 22, in parse_visit_date
    |     return int(year), int(month), int(day)
    |            ~~~^^^^^^
    | ValueError: invalid literal for int() with base 10: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py", line 30, in test_parse_visit_date_no_crash
    |     assert False, f"parse_visit_date crashed on {s!r}: {type(e).__name__}: {e}"
    | AssertionError: parse_visit_date crashed on '--': ValueError: invalid literal for int() with base 10: ''
    | assert False
    | Failing test case: test_parse_visit_date_no_crash(
    |     s='--',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py", line 28, in test_parse_visit_date_no_crash
    |     result = parse_visit_date(s)
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/museum_admission_parser.py", line 21, in parse_visit_date
    |     year, month, day = text.strip().split('-')
    |     ^^^^^^^^^^^^^^^^
    | ValueError: not enough values to unpack (expected 3, got 1)
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py", line 30, in test_parse_visit_date_no_crash
    |     assert False, f"parse_visit_date crashed on {s!r}: {type(e).__name__}: {e}"
    | AssertionError: parse_visit_date crashed on '': ValueError: not enough values to unpack (expected 3, got 1)
    | assert False
    | Failing test case: test_parse_visit_date_no_crash(
    |     s='',  # or any other generated value
    | )
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py", line 28, in test_parse_visit_date_no_crash
    |     result = parse_visit_date(s)
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/museum_admission_parser.py", line 21, in parse_visit_date
    |     year, month, day = text.strip().split('-')
    |                        ^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'strip'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/workspace/L6_L7_54_museum_admission/LAB7/test_museum_admission_parser.py", line 30, in test_parse_visit_date_no_crash
    |     assert False, f"parse_visit_date crashed on {s!r}: {type(e).__name__}: {e}"
    | AssertionError: parse_visit_date crashed on None: AttributeError: 'NoneType' object has no attribute 'strip'
    | assert False
    | Failing test case: test_parse_visit_date_no_crash(
    |     s=None,  # or any other generated value
    | )
    +-----------------------------------
========================== short test summary info ===========================
FAILED test_museum_admission_parser.py::test_format_visitor_name_no_crash - E...
FAILED test_museum_admission_parser.py::test_parse_group_ages_no_crash - Exce...
FAILED test_museum_admission_parser.py::test_parse_visit_date_no_crash - Exce...
============================== 3 failed in 1.98s ==============================
/workspace/L6_L7_54_museum_admission/LAB7$
```

SS3: Corrected application

python museum_admission_parser.py (invalid input, then valid input)

```
/workspace/L6_L7_54_museum_admission/LAB7$ python museum_admission_parser.py
==== Running format_visitor_name ====
Enter a visitor's full name (e.g. asha rao):
Name on ticket: Invalid name, please enter at least one word.

==== Running parse_group_ages ====
Enter the ages in the group separated by spaces (e.g. 34 12 70): 34 twelve 70
Group ages: Invalid ages, use whole numbers from 0 to 120.

==== Running parse_visit_date ====
Enter the visit date as YYYY-MM-DD (e.g. 2026-11-05): 2026-02-30
Visit date (year, month, day): Invalid date, use a real date as YYYY-MM-DD.
/workspace/L6_L7_54_museum_admission/LAB7$ python museum_admission_parser.py
==== Running format_visitor_name ====
Enter a visitor's full name (e.g. asha rao): asha rao
Name on ticket: Rao, Asha

==== Running parse_group_ages ====
Enter the ages in the group separated by spaces (e.g. 34 12 70): 34 12 70
Group ages: [34, 12, 70]

==== Running parse_visit_date ====
Enter the visit date as YYYY-MM-DD (e.g. 2026-11-05): 2026-11-05
Visit date (year, month, day): (2026, 11, 5)
/workspace/L6_L7_54_museum_admission/LAB7$
```

SS4: Fuzz tests passing on the corrected application

pytest test_museum_admission_parser.py

```
/workspace/L6_L7_54_museum_admission/LAB7$ pytest test_museum_admission_parser.py
============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-9.1.1, pluggy-1.6.0
rootdir: /workspace/L6_L7_54_museum_admission/LAB7
plugins: platformdirs-4.13.3, anyio-4.15.1, hypothesis-6.170.15
collected 3 items

test_museum_admission_parser.py ...                                      [100%]

============================== 3 passed in 1.48s ===============================
/workspace/L6_L7_54_museum_admission/LAB7$
```

Bugs found and fixes

Function	Input found by fuzzing	Crash	Fix
format_visitor_name	'' or only spaces	IndexError (parts[-1] on an empty list)	Return None when there are no words
format_visitor_name	None	AttributeError (None has no split)	Return None for non-string input
parse_group_ages	':', 'twelve'	ValueError from int()	Accept only plain digits 0 to 120, else return None
parse_group_ages	None	AttributeError	Return None for non-string input
parse_visit_date	'', '--', '2026-11'	ValueError (wrong number of parts or empty part)	Match YYYY-MM-DD exactly, else return None
parse_visit_date	None	AttributeError (None has no strip)	Return None for non-string input

Two problems that did not crash were also fixed: a single-word name printed as "Asha, " with a trailing comma, and impossible dates such as 2026-02-30 were accepted. Ages like '-5', '1.5' and '²' are now rejected.

Reflection

1. What input validation bugs did fuzzing reveal, and which inputs caused them?
Every function assumed well-formed text. Hypothesis found that all three crash on None, that an empty or blank name crashes format_visitor_name, that any non-numeric age such as ':' crashes parse_group_ages, and that empty or badly shaped dates such as '' and '--' crash parse_visit_date. It also shrank each failure to the smallest input that reproduces it, which made the cause obvious.

2. How did you fix the bugs?
Each function now checks its input before using it: it returns None for non-string input, checks that there is something to parse, and validates the format (plain digits for ages, YYYY-MM-DD plus a real calendar date for visit dates). main() prints a clear message instead of a traceback. After the fix all three Hypothesis tests pass, and a further 20,000 random inputs per function caused no crash.

3. Why does fuzz testing find bugs that hand-written test cases miss?
Hand-written tests use the inputs the developer already expects, so they share the same assumptions as the code. Fuzzing generates hundreds of unexpected inputs (empty strings, None, symbols, unicode) automatically and keeps the ones that fail. It is good at finding crashes, but it only checks the properties the test asserts, so it complements rather than replaces normal unit tests for correct output.

museum_admission_parser.py (corrected)

```
import re
from datetime import date

MAX_AGE = 120
_AGE_PATTERN = re.compile(r"[0-9]+")
_DATE_PATTERN = re.compile(r"([0-9]{4})-([0-9]{1,2})-([0-9]{1,2})")


def format_visitor_name(name):
    """
    Formats a full name as 'Surname, Given names' for the ticket.
    Returns None if name is not a string or has no words.
    """
    if not isinstance(name, str):
        return None
    parts = name.split()
    if not parts:
        return None
    surname = parts[-1].title()
    given = ' '.join(parts[:-1]).title()
    # A single-word name has no given names, so no trailing comma
    return f"{surname}, {given}" if given else surname


def parse_group_ages(text):
    """
    Parses space-separated ages into a list of ints.
    Returns None if text is not a string, is empty, or any value is not a
    whole number from 0 to MAX_AGE.
    """
    if not isinstance(text, str):
        return None
    values = text.split()
    if not values:
        return None
    ages = []
    for value in values:
        # Only plain ASCII digits: rejects '-5', '1.5', '²' and other
        # characters that str.isdigit() or int() would treat differently
        if not _AGE_PATTERN.fullmatch(value):
            return None
        age = int(value)
        if age > MAX_AGE:
            return None
        ages.append(age)
    return ages


def parse_visit_date(text):
    """
    Parses a YYYY-MM-DD date into a (year, month, day) tuple.
    Returns None if text is not a string or not a real calendar date.
    """
    if not isinstance(text, str):
        return None
    match = _DATE_PATTERN.fullmatch(text.strip())
    if not match:
        return None
    year, month, day = (int(part) for part in match.groups())
```

```
    try:
        date(year, month, day)  # rejects 2026-02-30, month 13, year 0
    except ValueError:
        return None
    return year, month, day


def main():
    print("==== Running format_visitor_name ====")
    value = input("Enter a visitor's full name (e.g. asha rao): ")
    result = format_visitor_name(value)
    print("Name on ticket:", result if result is not None else "Invalid name, please enter at
least one word.")

    print("\n==== Running parse_group_ages ====")
    value = input("Enter the ages in the group separated by spaces (e.g. 34 12 70): ")
    result = parse_group_ages(value)
    print("Group ages:", result if result is not None else f"Invalid ages, use whole numbers
from 0 to {MAX_AGE}.")

    print("\n==== Running parse_visit_date ====")
    value = input("Enter the visit date as YYYY-MM-DD (e.g. 2026-11-05): ")
    result = parse_visit_date(value)
    print("Visit date (year, month, day):", result if result is not None else "Invalid date,
use a real date as YYYY-MM-DD.")


if __name__ == "__main__":
    main()
```

test_museum_admission_parser.py

```
from hypothesis import given, strategies as st
from museum_admission_parser import format_visitor_name, parse_group_ages, parse_visit_date


@given(st.text() | st.none())
def test_format_visitor_name_no_crash(s):
    try:
        result = format_visitor_name(s)
    except Exception as e:
        assert False, f"format_visitor_name crashed on {s!r}: {type(e).__name__}: {e}"
    # Valid input gives a string, invalid input gives None
    assert result is None or isinstance(result, str)


@given(st.text() | st.none())
def test_parse_group_ages_no_crash(s):
    try:
        result = parse_group_ages(s)
    except Exception as e:
        assert False, f"parse_group_ages crashed on {s!r}: {type(e).__name__}: {e}"
    # Valid input gives a list of whole-number ages, invalid input gives None
    assert result is None or all(isinstance(age, int) and age >= 0 for age in result)


@given(st.text() | st.none())
def test_parse_visit_date_no_crash(s):
    try:
        result = parse_visit_date(s)
    except Exception as e:
        assert False, f"parse_visit_date crashed on {s!r}: {type(e).__name__}: {e}"
    # Valid input gives a (year, month, day) tuple, invalid input gives None
    assert result is None or (isinstance(result, tuple) and len(result) == 3)
```